

MongoDB Aggregation — Real-World Case Studies

Practice Problems • sample_mflix.movies collection • No Solutions

Instructions: Solve each case study using MongoDB Compass Aggregations. Use only the **movies** collection. Do not use the solutions or backend code; focus on designing the aggregation pipeline yourself.

Case Study 1 — Movie Discovery by Release Year

Real-world context: You are building a movie discovery platform. Users want to browse movies released in a particular year.

Problem statement: Find all movies that were released in 2010. Return the matching movie documents.

Case Study 2 — Latest Movies

Real-world context: A streaming platform wants to display recently released movies first.

Problem statement: Find movies released after the year 2000 and arrange them from newest release year to oldest. Return all matching results.

Case Study 3 — Top 10 Newest Movies

Real-world context: The homepage of a movie platform has a section called “Latest Movies” and can display only 10 movies.

Problem statement: Find the 10 newest movies in the collection based on release year.

Case Study 4 — Movie Listing Pagination

Real-world context: A movie website displays 10 movies on each page. A user has opened page 3.

Problem statement: Create an aggregation pipeline that returns the movies belonging to page 3. Assume the movies should be displayed from newest to oldest.

Challenge: Try to solve this without looking up a ready-made pipeline. First identify what the database needs to do, then decide the aggregation stages.

Case Study 5 — Action Movie Discovery

Real-world context: A streaming platform allows users to browse movies by genre. A user selects the Action genre.

Problem statement: Find all movies that belong to the Action genre.

Case Study 6 — Most Popular Movie Genres

Real-world context: A movie analytics team wants to understand which genres appear most frequently in the movie database.

Problem statement: Find the number of movies associated with each genre and arrange the genres from the most common to the least common.

Challenge: Try to solve this without looking up a ready-made pipeline. First identify what the database needs to do, then decide the aggregation stages.

Case Study 7 — Average Runtime by Genre

Real-world context: A streaming company wants to understand the typical movie length for every genre.

Problem statement: Calculate the average runtime of movies for each genre and arrange the genres according to their average runtime.

Challenge: Try to solve this without looking up a ready-made pipeline. First identify what the database needs to do, then decide the aggregation stages.

Case Study 8 — Movies Released Per Year

Real-world context: A movie analytics dashboard needs to show how many movies were released in each year.

Problem statement: Calculate the total number of movies for every release year and arrange the years from the year with the most movies to the year with the fewest movies.

Case Study 9 — Highly Rated Movies

Real-world context: A movie recommendation platform wants to create a list of highly rated movies using IMDb ratings.

Problem statement: Find movies with an IMDb rating greater than 8. Arrange them from highest IMDb rating to lowest and return only the top 10.

Case Study 10 — Complete Movie Listing Pagination

Real-world context: You are developing a Netflix-style movie listing page. The page should show only movies with an IMDb rating above 7. Movies must be displayed from the highest IMDb rating to the lowest. The UI displays 10 movies per page, and the user is currently on page 3.

Problem statement: Build an aggregation pipeline that returns exactly the movies required for page 3 while respecting the rating filter, sorting requirement, and page size.

Challenge: Try to solve this without looking up a ready-made pipeline. First identify what the database needs to do, then decide the aggregation stages.

Student Challenge Checklist

#	Skill to identify
1	Filtering documents
2	Filtering + sorting
3	Sorting + limiting results
4	Pagination using page number and page size
5	Filtering array values
6	Working with arrays and aggregation
7	Grouping and calculating averages
8	Grouping and counting
9	Nested-field filtering + sorting + limiting
10	Combining filtering, sorting, skipping and limiting for real-world pagination

Dataset: MongoDB sample_mflix → movies. The exercises are intentionally solution-free so they can be used as classroom activities.